Deteção de Autoria em Texto: Humanos vs IA

Abordagens de Machine Learning e Deep Learning

Carlos Tiago da Costa Ribeiro – A65433
Oleksii Tantsura – A102131
Vicente de Carvalho Castro – PG60395
Martim de Oliveira e Melo Ferreira – PG60391

Construção do Dataset

	Classe	Fontes
	Humano	HC3, M4, Wikipedia
	OpenAI	HC3, ai-texto-detection-pile
	Google	OpenTuringBench, M4
	Meta	OpenTuringBench, M4
	Anthropic	Geração sintética (Claude API)/lmsys-chat-1m

Fontes de Dados Brutos

↓

Filtro de Qualidade
Restrição 70 – 130 palavras

↓

Dataset Balanceado
~800 exemplos por classe

Distribuição e Estimação do Erro

Metodologia de Validação

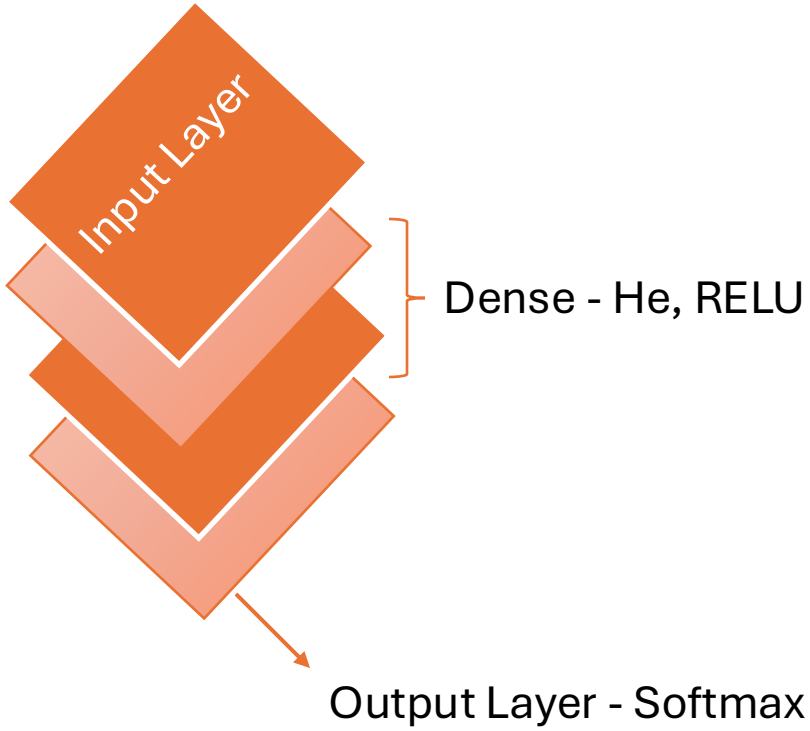
Hold-out validation

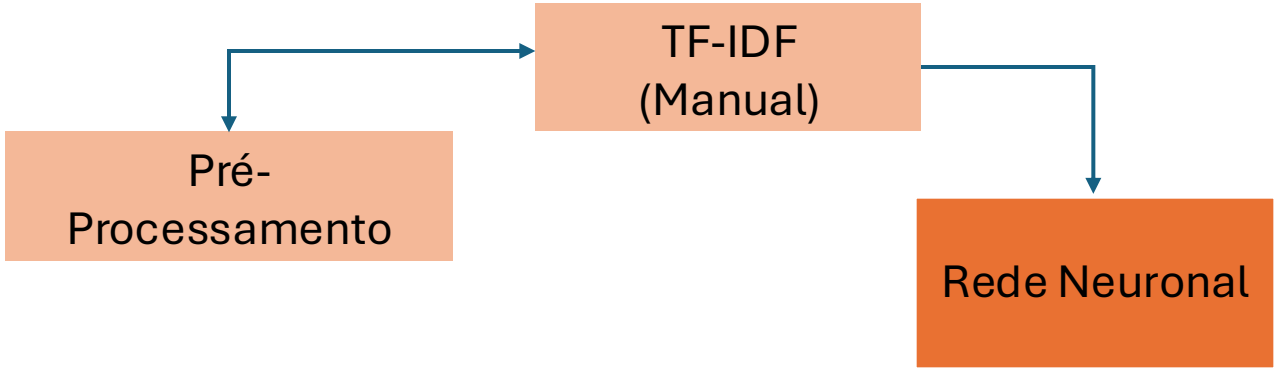
Com “random_state = 42” para garantir reprodutividade

Métricas

Accuracy	F1-Score
Avaliação de performance global	Macro-averaging para penalizar desequilíbrios não detetados

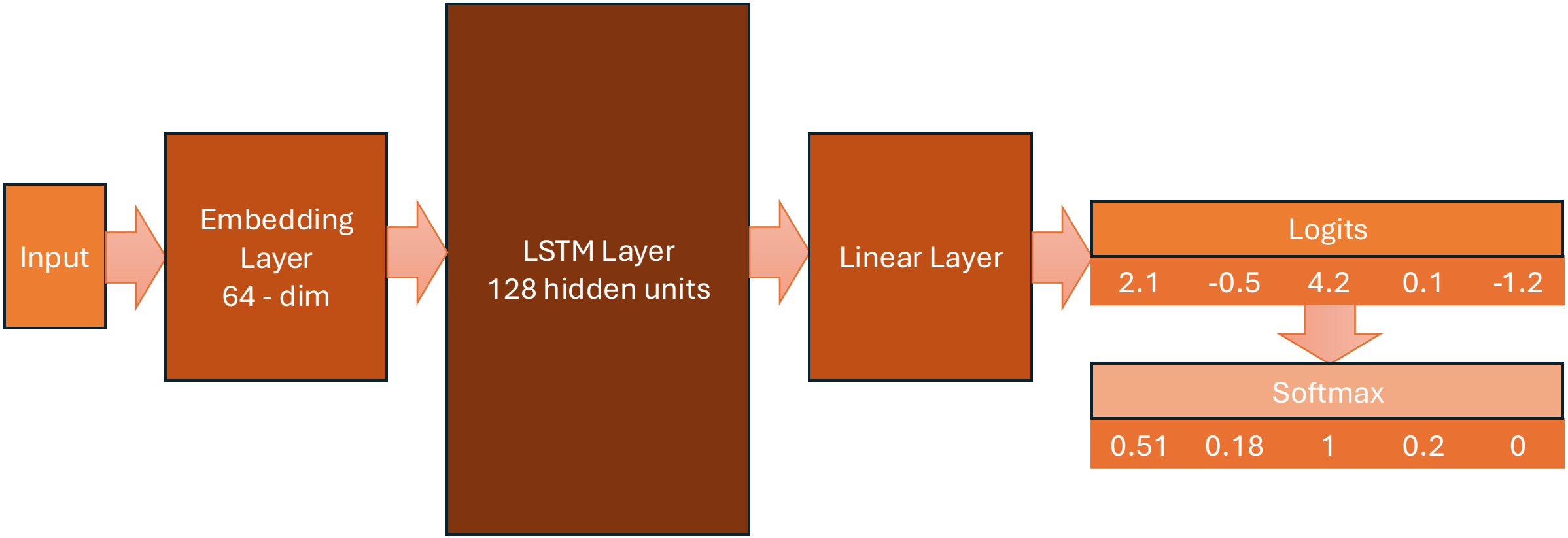
Implementação "De Raiz" - Numpy



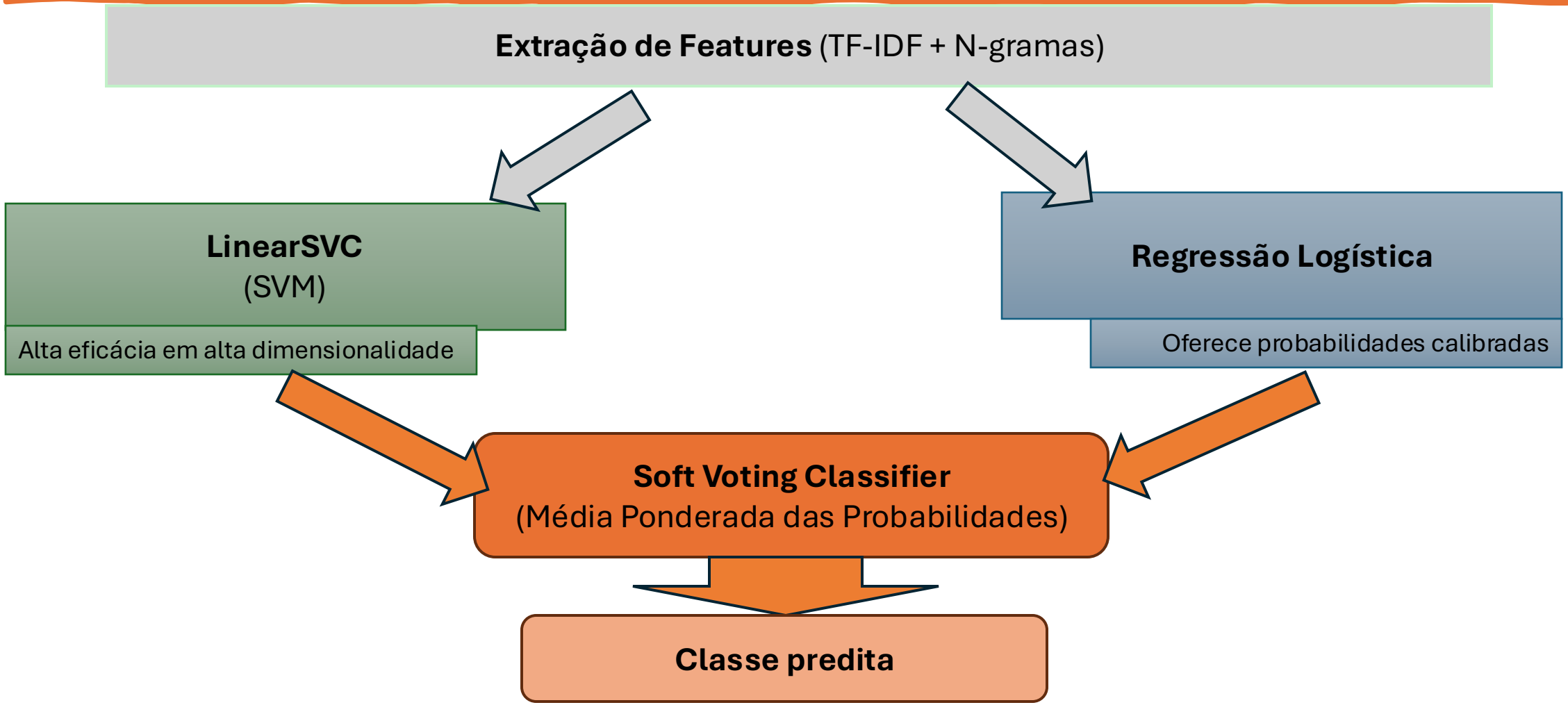


Funções Implementadas	Sigmoid, RELU, Softmax, Cross_entropy_loss
Otimização	Backpropagation com atualização de gradientes

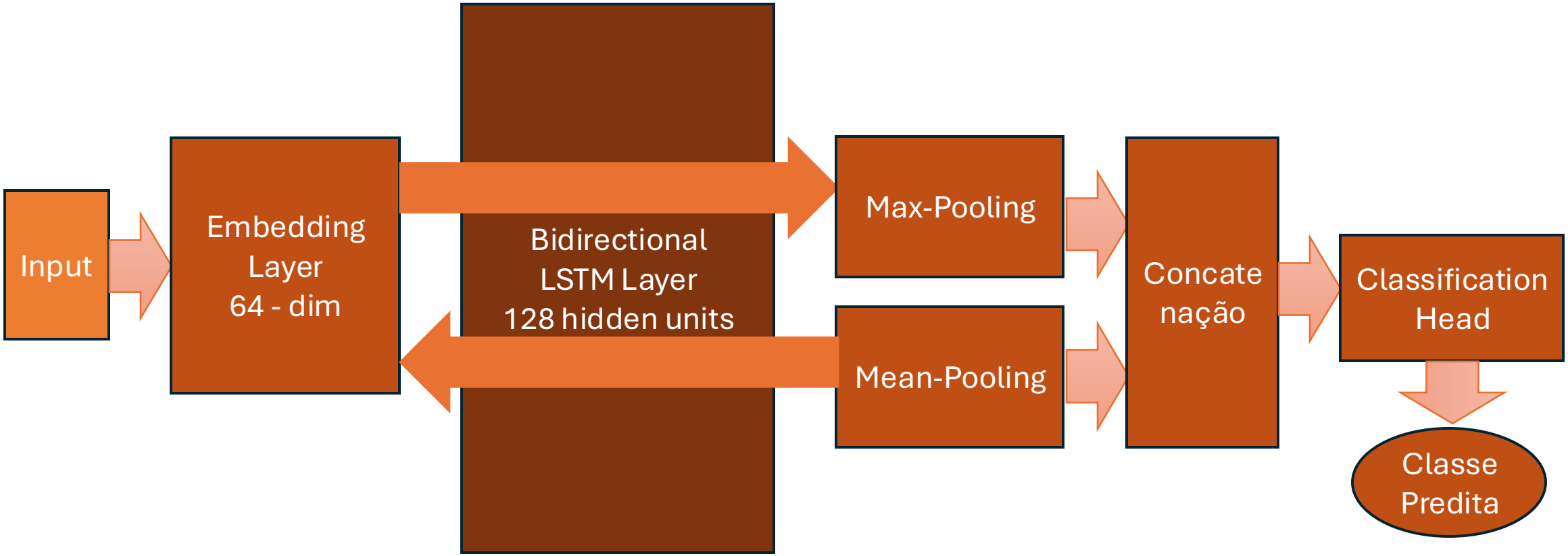
LSTM Classifier - PyTorch



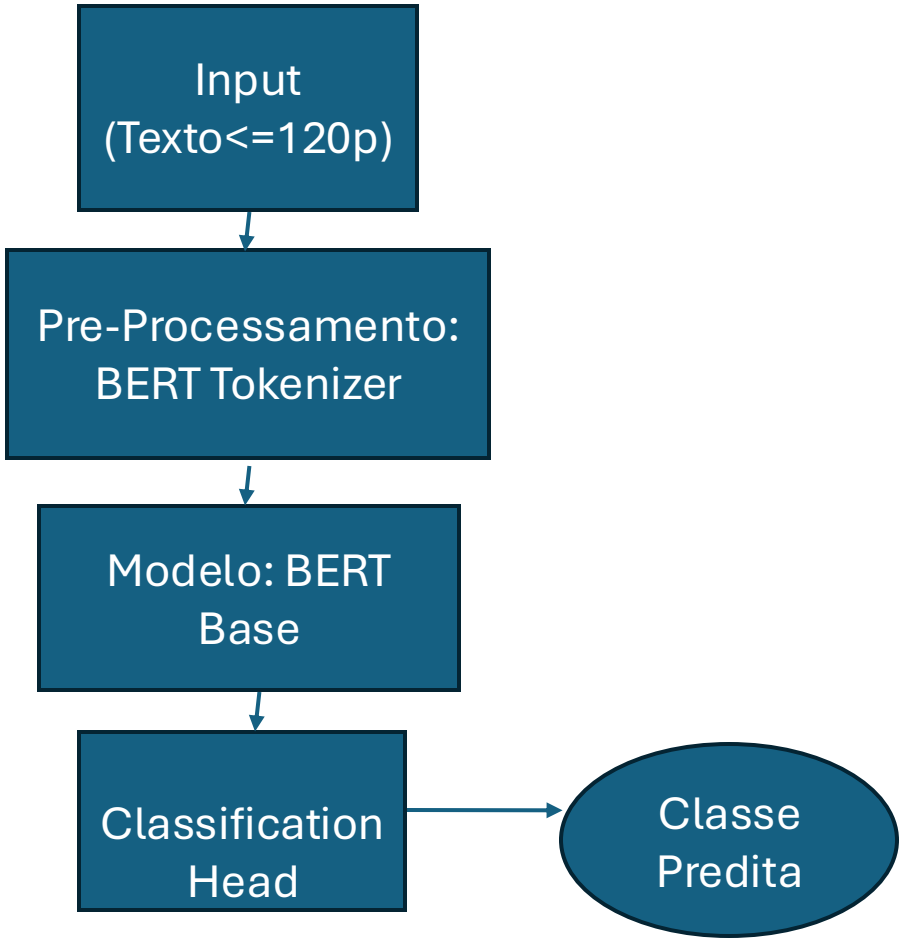
Ensemble Híbrido



Bidirectional LSTM



Implementação Transformer – BERT (Hugging Face)



Categoria	Detalhes da Implementação
Modelo & Tokenizer	bert-base-uncased (Truncamento estrito a 120 palavras / 200 Tokens)
Arquitetura (Fine-Tuning)	Layer Freezing: 6 primeiras camadas do BERT congeladas para evitar overfitting.
Função de Perda (Loss)	Weighted Cross-Entropy Loss: Pesos 3x maiores para Meta e Google.
Otimização	AdamW (LR: 2e-5), Linear Scheduler c/ Warmup (10%), Gradient Clipping.
Avaliação / Métricas	Mid-Epoch Checkpointing: Avaliação no dataset do professor a cada 100 passos.

Resultados

Submissão 1:

- **Regressão Logística**
  - Train acc: 1.00
  - Val acc: 0.12
- **DNN (NumPy)**
  - Train acc: 1.00
  - Val acc: 0.32
  - Early stopping (epoch 97)

- Epoch 10 → acc: 0.24
- Epoch 20 → acc: 0.08
- Early stopping @ epoch 22

Resultados

Submissão 2:

- LinearSVC: **0.878 ± 0.011**
- Logistic Regression: **0.835 ± 0.008**

```
  Avg training loss: 0.1734
  Evaluating on Validation set...
  Validation accuracy: 0.9838


==================================================
Final Evaluation on Test Set
==================================================
Test accuracy: 0.9843
              precision    recall  f1-score   support

       Human       1.00      0.98      0.99      4500
   Anthropic       0.99      0.98      0.99      2213
      Google       0.99      0.98      0.99      2250
        Meta       0.98      0.99      0.99      2218
      OpenAI       0.95      0.99      0.97      2247

    accuracy                           0.98     13428
   macro avg       0.98      0.99      0.98     13428
weighted avg       0.98      0.98      0.98     13428
```

```
# WEIGHTED LOSS: [Human, Anthropic, Google, Meta, OpenAI]
# Giving 3x importance to Google and Meta
class_weights = torch.tensor([1.0, 1.0, 3.0, 3.0, 1.0]).to(DEVICE)
criterion = torch.nn.CrossEntropyLoss(weight=class_weights)
```

Trabalho futuro - Few-Shot Learning com SetFit

O modelo (`bge-large`) gera pares de frases para aprender as diferenças subtis de estilo (Loss).

Treino de um classificador rápido em cima dos embeddings gerados.

Previsão Final (5 Classes) -> Acurácia: 80.0%

Parâmetro	Detalhes da Implementação
Modelo Base	BAAI/bge-large-en-v1.5 (~335M Parâmetros)
Divisão de Dados	Treino: 205 exemplos / Validação: 30 exemplos (Fornecidos pelo Professor)
Context Window	max_length = 512 (Garante a leitura integral de cada texto)
Hiperparâmetros	batch_size = 8, num_epochs = 1, seed = 42 (Para evitar overfitting)
Resultado Obtido	80.0% de Acurácia no conjunto de validação (24 em 30 corretos)